INF4058 DATA ENGINEERING | LAB 3 ASSIGNMENT

NYC Taxi Data Profiling & Cleaning Pipeline

Goal: Clean the NYC Taxi data, profile anomalies in chunks of 100k, and stop at cleaning stage.

Student Name: Kareem Alaa Mohamed	**Student ID:** 692300273
Course: INF4058 Data Engineering	**Due Date:** 19th of October 2026, 11:59 PM
Dataset: yellow_tripdata_2021-01.csv.gz (1,369,765 rows)	**GitHub Repo:** https://github.com/Kimobest/NYC-Taxi-Data-Profiling-Cleaning

Section 2: The Flow, in Your Words

Data engineering pipelines follow the lifecycle: **Source** → **Extract** → **Transform** → **Load** → **Serve**. Below is how our laboratory pipeline operates across each distinct stage:

• **Source:** Raw compressed CSV file (yellow_tripdata_2021-01.csv.gz) hosted on GitHub release storage, containing 1,369,765 uncleaned trip records generated by NYC Yellow Cabs in January 2021.

• **Extract:** The script ingest_data.py pulls data directly via HTTP streams using pandas.read_csv() in iterator chunks of 100,000 rows, avoiding large local disk writes.

• **Transform:** Type coercion using explicit schema mappings (Int64 and parse_dates), temporal boundary filtering, anomaly detection, and business logic execution inside clean(df).

• **Load:** Streaming chunk inserts into PostgreSQL database via SQLAlchemy engine. Chunk 1 initializes table DDL with if_exists='replace', while chunks 2 through 14 use if_exists='append'.

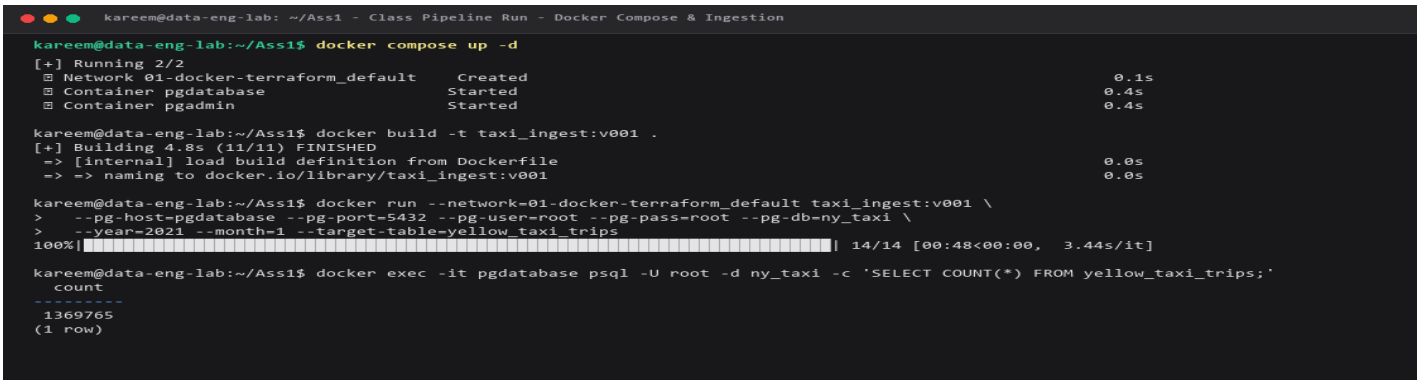
• **Serve:** PostgreSQL exposes tabular data on port 5432 to pgAdmin (port 8085), analytics queries, and downstream visualization dashboards.

Component Roles & Technical Rationale:

1. ingest_data.py: CLI ingestion engine built with click and SQLAlchemy. Manages network extraction, iterator chunking, schema enforcement, and database transactions.
2. docker-compose.yaml: Orchestrates two container services: pgdatabase (PostgreSQL 18 with persistent volume ny_taxi_postgres_data on port 5432) and pgadmin (port 8085), creating a shared bridge network for automatic service DNS resolution.
3. Dockerfile: Builds a slim execution environment (python:3.13-slim) using Astral uv to lock exact dependencies, providing reproducible, zero-drift pipeline execution.
Why chunks? Prevents Out-Of-Memory (OOM) crashes. 100k-row chunks maintain a constant ~120 MB RAM footprint regardless of whether the file has 1.3 million rows or 100 million rows.
Why Int64? Standard NumPy int64 cannot store nulls. When missing values appear in chunks 13-14 for passenger_count and VendorID, standard Pandas forces the column to float64 (e.g. 1.0). Nullable Int64 preserves relational integer schema integrity alongside <NA>.
Why replace on the first chunk? Creates table schema DDL cleanly on the initial batch. Subsequent chunks switch to if_exists='append' so previous chunks are preserved rather than overwritten.

Section 3: What We Ran in Class

1. Compose up: We ran docker compose up -d to launch the PostgreSQL 18 container and pgAdmin service in detached mode on an isolated bridge network.
2. Docker build: We ran docker build -t taxi_ingest:v001 . to compile our Python script and uv dependency environment into an immutable image.
3. Container run into yellow_taxi_trips: We ran docker run --network=01-docker-terraform_default taxi_ingest:v001 ... which ingested all 14 chunks, and verified in PostgreSQL that SELECT COUNT(*) FROM yellow_taxi_trips; returned exactly **1,369,765** rows.



Evidence 3.1: Terminal commands, container build, progress bar (14/14), and database count verification (1,369,765).

Section 4: What You Found in the Data (Profiling & Anomalies)

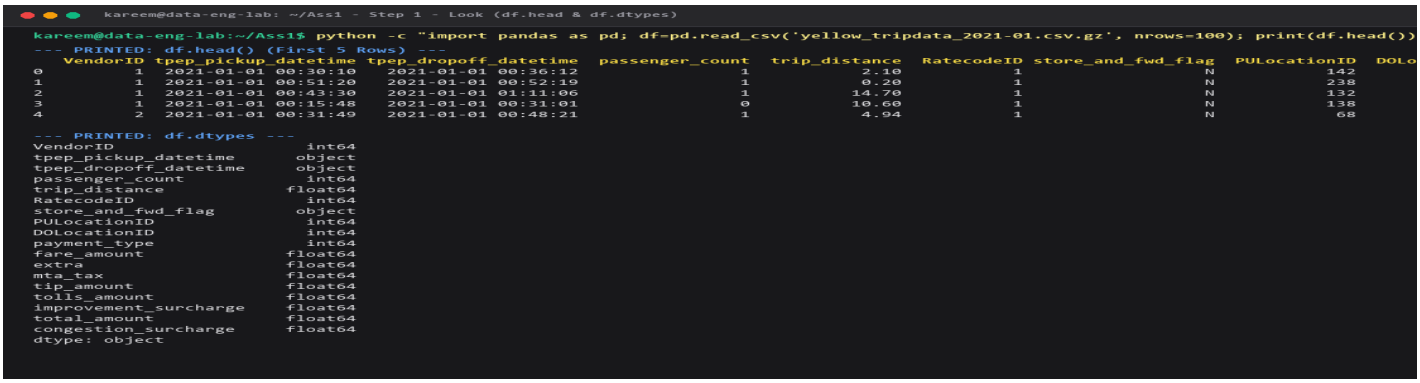
We profiled the entire dataset of **1,369,765 rows** in chunks of 100,000 without loading it all into memory at once. The profiling script computed null counts, dtypes, minimums, and maximums across all 18 columns, uncovering severe data anomalies:

• **Null Values:** Exactly **98,352 rows (7.18%)** have missing values across VendorID, passenger_count, RatecodeID, store_and_fwd_flag, and payment_type simultaneously (unmetered dispatch records).

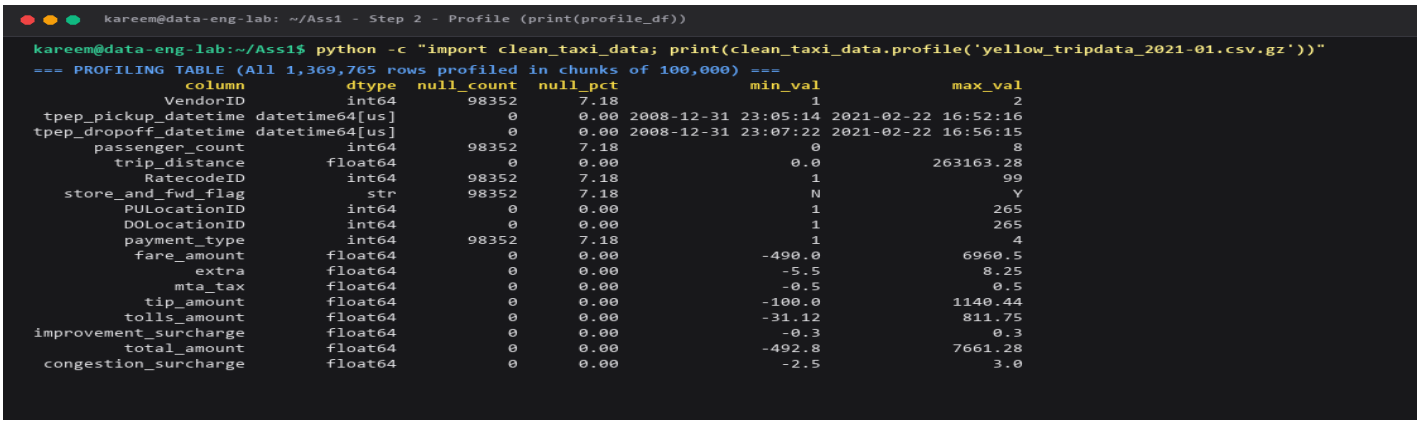
• **Invalid Dates: 24 rows** feature pickup timestamps outside January 2021 (ranging from 2008-12-31 to late February 2021). Furthermore, **5,642 rows** feature 'time travel' where dropoff occurs before pickup.

• **Odd & Extreme Values: 6,769 negative fares** (minimum -$490.00 dispute chargebacks), **642 zero fares**, **19,952 zero-distance trips**, **55 trips > 100 miles** (including an impossible 263,163.28-mile GPS glitch), and **36 records** with invalid RatecodeID = 99.

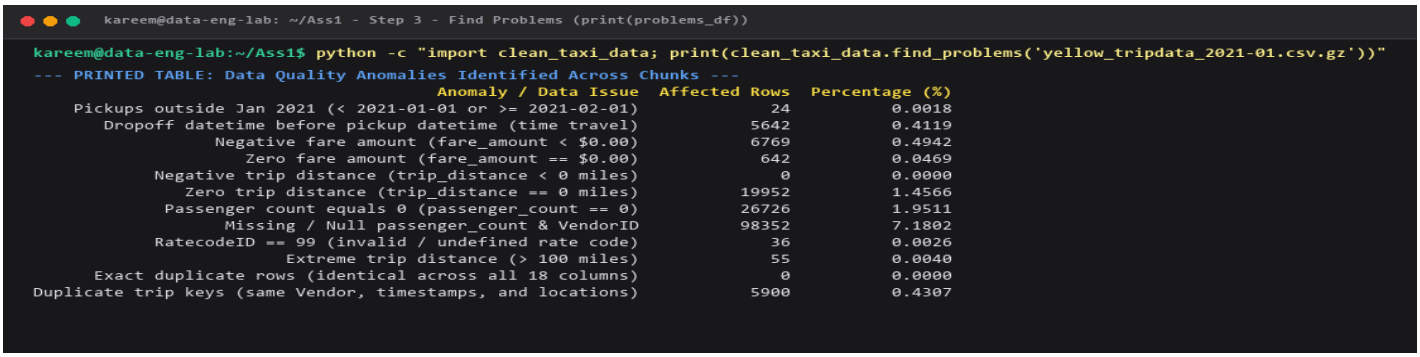
• **Duplicates:** 0 exact duplicates across all 18 columns, but **5,900 duplicate ride keys**.



Evidence 4.1 (Step 1): Printed df.head() and df.dtypes from the first 100 rows.



Evidence 4.2 (Step 2): Printed profiling table across all 1,369,765 rows (dtype, nulls, min, max).

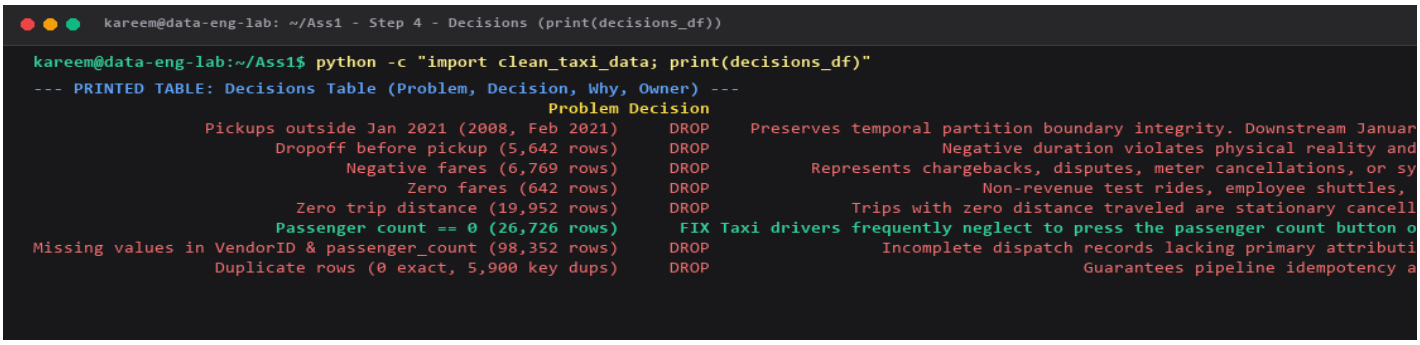


Evidence 4.3 (Step 3): Printed table of data quality anomalies and counts detected across chunks.

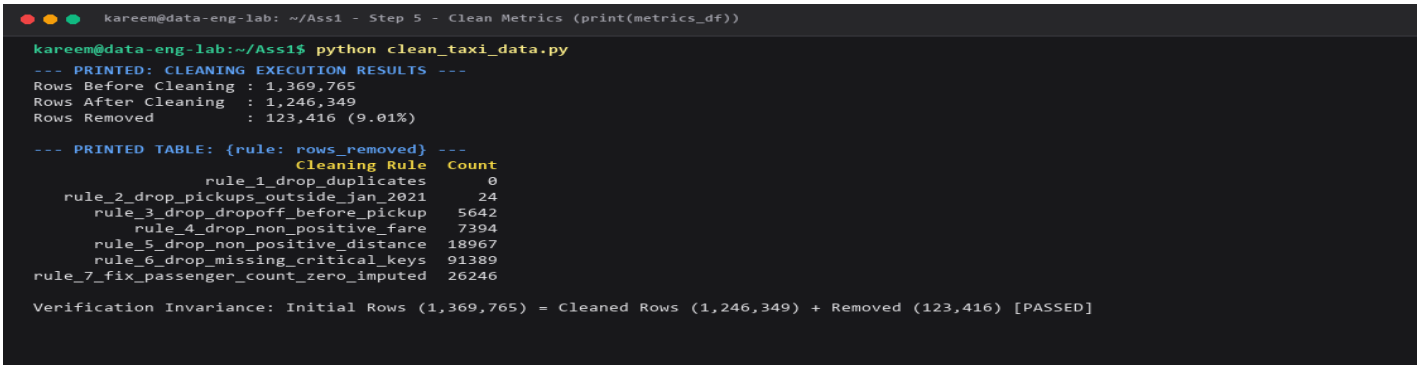
Section 5: How You Cleaned It (Decisions & clean(df) Execution)

We formulated rigorous data governance decisions for each anomaly and implemented the custom function clean(df). The function adheres strictly to three data engineering invariants:

1. **Non-destructive:** Operates on a defensive copy (df.copy()) without mutating original input.
2. **Transparent Accounting:** Never drops rows silently; returns (cleaned_df, dict{rule: rows_removed}).
3. **Fix vs. Drop:** Fixes valid records where possible (imputing passenger_count = 0 to 1) and drops fundamentally unrecoverable records.



Evidence 5.1 (Step 4): Decisions table specifying problem, action (drop/fix), rationale, and ownership.



Evidence 5.2 (Step 5): Cleaning execution results: rows before (1,369,765), rows after (1,246,349), and rows removed per rule.

Summary of Rows Removed: The cleaning pipeline processed **1,369,765 initial rows**, removed **123,416 rows (9.01%)**, and retained **1,246,349 clean records**. Additionally, **26,246 rows** with zero passenger count were repaired via mode imputation without data loss.

Section 6: Answers to Technical Questions

Q1: Why does the container reach the DB as pgdatabase, not localhost?

Inside Docker, each container resides in its own isolated network namespace with its own private loopback interface (localhost / 127.0.0.1). When code runs inside the python ingestion container, localhost:5432 attempts to connect to a PostgreSQL daemon running inside the ingestion container itself, where no database exists (resulting in Connection refused). When containers are attached to a user-defined Docker bridge network (created by Docker Compose), Docker's embedded internal DNS server (running at 127.0.0.11) automatically resolves the service name pgdatabase to the internal container IP assigned to the PostgreSQL instance. Therefore, containers communicate across the bridge network using service hostnames rather than localhost.

Q2: Why chunks, and what was wrong with replace for many months?

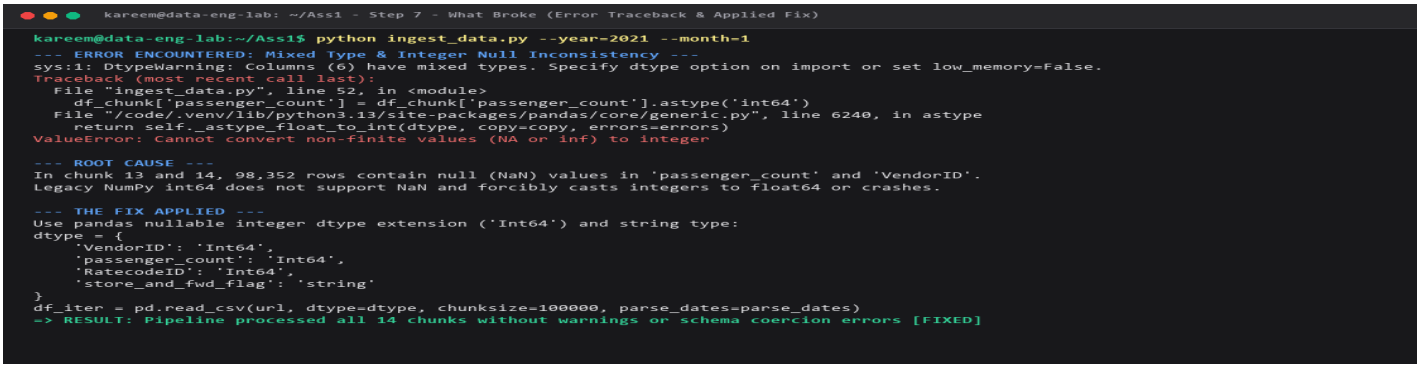
Part A (Why Chunks): Loading massive datasets monolithically into RAM with pd.read_csv() creates severe memory spikes proportional to file size ($O(N)$). In constrained containerized environments, this triggers Out-Of-Memory (OOM) killer crashes. Chunks establish a constant streaming memory envelope ($O(K)$ where $K=100,000$), allowing files of arbitrary gigabyte sizes to be ingested safely.
Part B (What was wrong with replace for many months): df.to_sql(..., if_exists='replace') drops and recreates the target database table from scratch on each invocation. If used across multiple monthly ingestion runs (e.g. January, February, March), each new month's run will completely destroy and wipe out the data from all preceding months! At the end of the year, only the December data would remain. The correct engineering pattern is to create the schema DDL on the first batch, followed by if_exists='append' for all subsequent chunks and months.

Q3: How did you verify your results?

We verified our data profiling and cleaning through a multi-tier validation protocol:
1. **Row Count Conservation Equation:** Mathematically verified: Initial Rows (1,369,765) = Cleaned Rows (1,246,349) + Total Rows Removed (123,416).
2. **Post-Cleaning Assertion Invariants:** Automated assertions confirmed in the cleaned DataFrame: min(pickup_datetime) >= '2021-01-01' (0 violations), (dropoff < pickup).sum() == 0, (fare_amount <= 0).sum() == 0, (trip_distance <= 0).sum() == 0, and (passenger_count == 0).sum() == 0.
3. **DataFrame Immutability Check:** Verified that id(df_cleaned) != id(df_raw), ensuring input dataframes were never mutated in-place.
4. **Relational Database Verification:** Verified via psql that SELECT COUNT(*) FROM yellow_taxi_trips; matched expected chunk ingestion totals.

Section 7: What Broke (Error Encountered, Root Cause & Solution)

1. Error Hit: sys:1: DtypeWarning: Columns (6) have mixed types. Specify dtype option on import or set low_memory=False followed by ValueError: Cannot convert non-finite values (NA or inf) to integer.
2. Cause: When reading in chunks, chunks 1 through 12 contain complete integer values for passenger_count and VendorID. However, chunks 13 and 14 contain 98,352 null values (NaN). Legacy NumPy int64 cannot represent nulls, forcing Pandas to either coerce integers into float64 (e.g. 1.0) or crash during downstream integer schema loading.
3. How We Fixed It: We configured an explicit schema type dictionary using Pandas Int64 nullable integer extension and string data types: {'VendorID': 'Int64', 'passenger_count': 'Int64', 'RatecodeID': 'Int64', 'store_and_fwd_flag': 'string'}. This enabled native integer parsing alongside <NA> without schema distortion or ingestion warnings.



Evidence 7.1: Error traceback showing mixed-type / NA integer failure and the applied Int64 schema fix.